Tiered Agent Memory

Giving AI coding agents an institutional memory

A context-engineering pattern we landed on.

The two problems

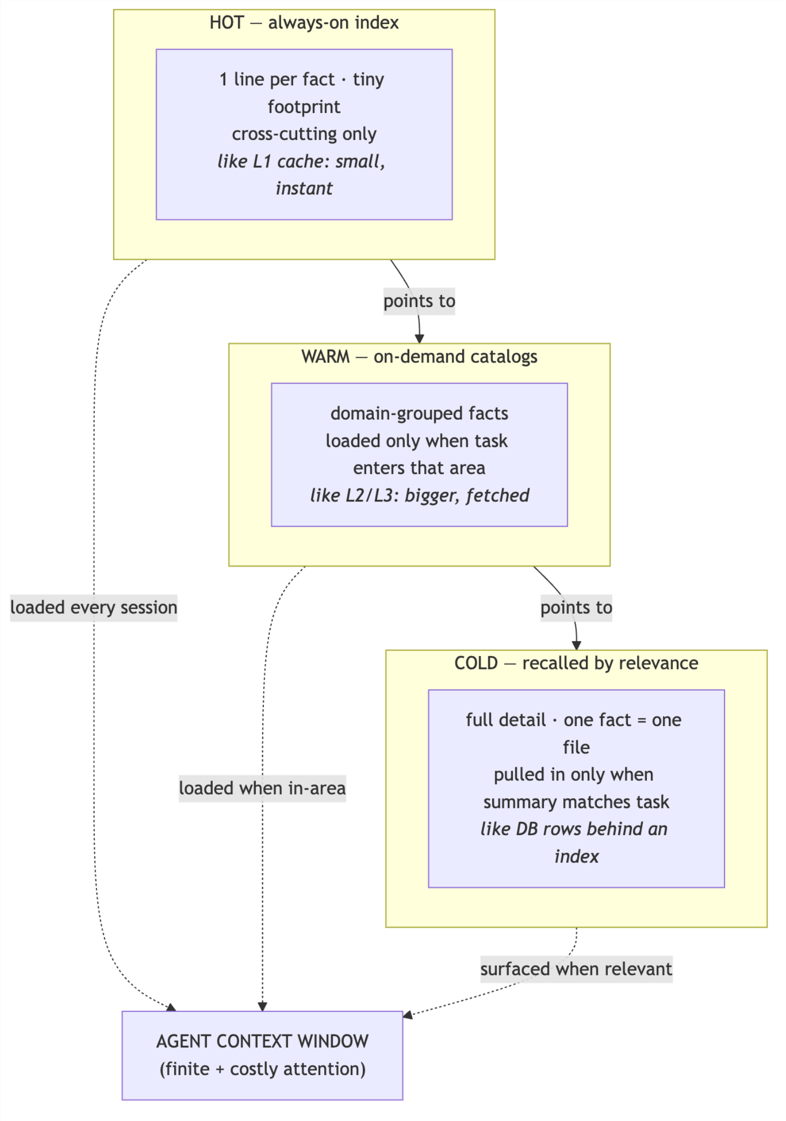
- AI agents are **amnesiac** — every session starts from zero, so the same constraints, preferences, and past mistakes get re-explained endlessly.
- Context is **finite and metered** — you *can't* paste everything you know into every session: it's expensive, slow, and dilutes the model's attention on the actual task.

> The tension: more knowledge available, but loading more makes the agent **worse and costlier**.

The insight

The same problem **hardware solved decades ago**: you can't keep everything fast and close — so you **tier** it.

`small / hot / always-on` → `larger / warm / on-demand` → `large / cold / fetched-when-relevant`


HOT — always-on index
1 line per fact · tiny footprint
cross-cutting only
like L1 cache: small, instant
points to
WARM — on-demand catalogs
domain-grouped facts
loaded only when task enters that area
like L2/L3: bigger, fetched
points to
COLD — recalled by relevance
full detail · one fact = one file
pulled in only when summary matches task
like DB rows behind an index
loaded every session
loaded when in-area
surfaced when relevant
AGENT CONTEXT WINDOW
(finite + costly attention)

Pillar 1 — Atomic, typed facts

- **One fact = one file.** Atomicity makes facts easy to update, delete, and dedupe — no giant doc nobody maintains.
- Each fact is **typed**: who-the-user-is · feedback/corrections (*with the why*) · ongoing-work context · external references.
- Type drives **policy**: how long a fact lives, how aggressively it's recalled.

Pillar 2 — Tiered loading *(the cost optimization)*

- **Hot index** — one line per fact, always loaded. Just enough to know a fact exists and when it's relevant. Deliberately tiny, cross-cutting only.
- **Warm catalogs** — domain-grouped, loaded only when the task enters that domain.
- **Cold recall** — full detail pulled in for a single fact only when its summary matches.

> “ This is where the **token + attention savings** come from. ”

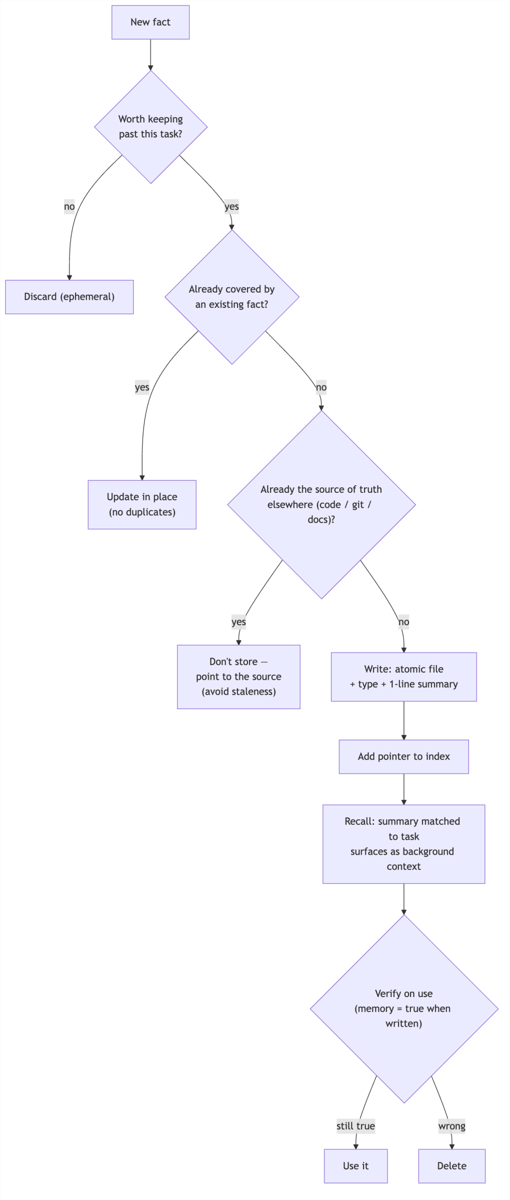
Pillar 3 — Relevance-based recall

- Each fact carries a one-line **description written for matching**, not for humans.
- The system surfaces **only relevant facts**...
- ...and surfaces them as **background context, not commands** — so old notes can't hijack new work.

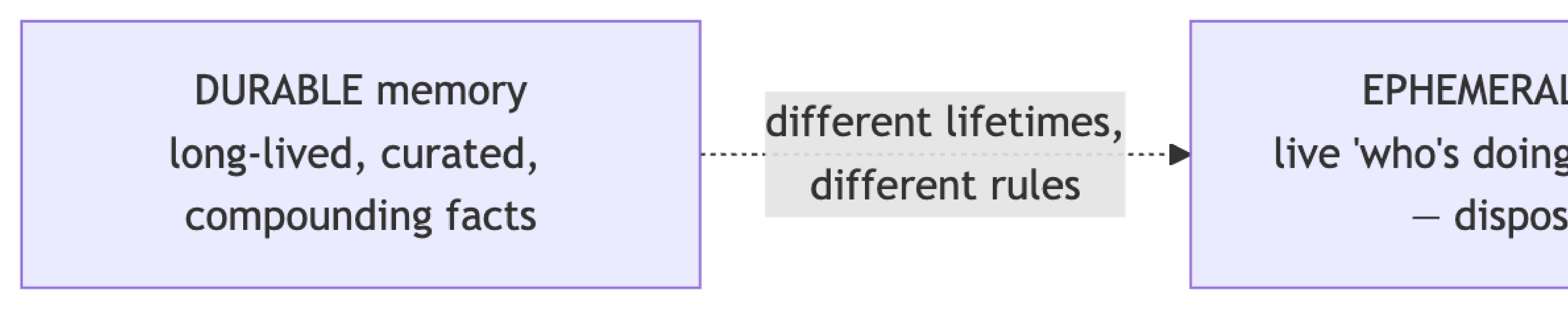
Pillar 4 — Curation discipline *(why it doesn't rot)*

The failure mode of every knowledge base is **staleness**. Four rules prevent it:

- **Dedup** — update the existing fact, never add a near-duplicate.
- **Delete-on-wrong** — a disproven fact is removed, not buried.
- **Single source of truth** — don't store what code/git/docs already record; point to it.
- **Verify-on-use + absolute dates** — a memory reflects what was true *when written*; confirm before acting, never store "yesterday."


New fact
Worth keeping past this task?
no
yes
Discard (ephemeral)
Already covered by an existing fact?
yes
no
Update in place (no duplicates)
Already the source of truth elsewhere (code / git / docs)?
yes
no
Don't store — point to the source (avoid staleness)
Write: atomic file + type + 1-line summary
Add pointer to index
Recall: summary matched to task surfaces as background context
Verify on use (memory = true when written)
still true
wrong
Use it
Delete

Pillar 5 — Separate durable vs ephemeral



- Long-lived knowledge and live "who's-doing-what-now" coordination have **different lifetimes and different rules**.
- Mixing them is how knowledge bases fill with **noise**. Keep two channels.

What changes in practice

- **Knowledge compounds** — a correction is captured once, applied every future session.
- **Lower context cost** — always-on footprint stays tiny vs. dumping everything.
- **More consistent behavior** — fewer repeated mistakes, less re-explaining.
- **Trustworthy over time** — curation prevents the stale-wiki death spiral.

> “ *Honest caveat:* it only works **with** the discipline. The rules are the product, not the files. ”

Why it generalizes

- Nothing here is tool-specific. It's **caching, database indexing, lazy-loading, and good onboarding docs** — applied to an agent's working memory.
- Transferable to **any agentic AI workflow, any team**.

"Here's the pattern — happy to go deeper on the mechanics."